

Running LLMs Locally Cheat Sheet

Irurueta Docs > Guides & References > AI > Running LLMs Locally

Baseline: Ollama 0.34.4 · llama.cpp v0.5.0 (b11239; llama serve = llama-server) · vLLM 0.30.0 · SGLang 0.5.20 · TGI v3.3.7 (maintenance) · LM Studio 0.4 · Docker Model Runner v1.2.8 · LocalAI v4.10.0 · mlx-lm 0.31.3 · LiteLLM 1.103.0 · Claude Code 2.1.284 · Copilot CLI 1.0.89 · Python 3.12 / openai 3.20.0 · Spring Boot 4.1 / Spring AI 2.0.1 · running example DocsAssistant on Ollama llama3.1:8b + nomic-embed-text. Checked 2026-09-28. No benchmark numbers or prices — measure your own. Quick reference only — verify against the official docs before production use.

ENGINE DECISION TREE

- Laptop, one user → Ollama / LM Studio; Apple silicon → + MLX-LM
- Docker workflow → Docker Model Runner (Compose models:)
- One GPU, every flag → llama.cpp (llama serve) or Ollama
- Multi-user GPU server → vLLM / SGLang (continuous batching, paged / radix KV)
- Enterprise + NVAIE licence → NVIDIA NIM
- > 1 client → put a gateway with auth in front

Why local: privacy & residency · cost at volume · offline · latency · control.
Trade-offs: quality gap · ops burden · hardware.

SIZING FORMULAS

$M_w \approx P \times b$ (b: FP16 2, Q8_0 ≈1.06, Q4_K_M ≈0.61 B)
 $M_{kv} = 2 \cdot L \cdot n_{kv} \cdot d_h \cdot T \cdot b \cdot B$
 $M \approx M_w + M_{kv} + 0$ (0 ≈ 1 GiB, measure)
 $tok/s_{max} \approx bandwidth / (P_{active} \cdot b + M_{kv})$

GiB @32k	FP16	Q8_0	Q4_K_M
8B dense	19.9	12.9	9.6
20B dense	44.3	26.8	18.4
70B dense	141.4	80.3	50.8
120B MoE	221.2	119.0	69.9

MoE: total params → memory; active → speed. Decode is bandwidth-bound; ollama ps → 100% GPU?

GGUF QUANT LADDER

Type	bits/w	Use
Q8_0	8.50	near-lossless
Q6_K	6.56	≈16-bit quality
Q5_K_M	5.70	good compromise
Q4_K_M	4.89	default (-hf)
IQ4_XS...	4.46–	low-bit; needs
IQ2_M	2.93	imatrix

$q = \text{round}(x/s) + z$; $\hat{x} = s(q - z)$

GPU servers: FP8 · AWQ · GPTQ · NVFP4 via llm-compressor (AutoAWQ deprecated). NF4 = normal quantiles (QLoRA). KV cache: -ctk q8_0 · OLLAMA_KV_CACHE_TYPE=q8_0 · -kv-cache-dtype fp8.

SECURITY CHECKLIST

- Most engines are **unauthenticated by default** (Ollama ignores keys; DMR API is unauthenticated; LM Studio / LocalAI auth off)
- Bind to 127.0.0.1 / private net; publish -p 127.0.0.1:... only
- TLS + per-client keys at a proxy / LiteLLM
- vLLM --api-key guards only /v1, /v2, /inference, /cohere
- Hide model-management routes (/api/pull, /api/delete, DMR)
- K8s NetworkPolicy: only gateway → engine; block egress
- No hosted fallback / OLLAMA_NO_CLOUD=1 when data must stay
- Secrets from env / vault, never in committed config

OLLAMA

```
ollama pull llama3.1:8b ollama run llama3.1:8b
ollama list · ps · stop · rm · create -f Modelfile
ollama run hf.co/<user>/<repo>-GGUF:Q4_K_M
OLLAMA_CONTEXT_LENGTH=32768 ollama serve
OLLAMA_KEEP_ALIVE=1h · OLLAMA_NUM_PARALLEL=4
```

```
FROM llama3.1:8b
PARAMETER num_ctx 16384
PARAMETER temperature 0.2
SYSTEM """"You are DocsAssistant""""
```

Default context is small (4k < 24 GiB VRAM) and overflow is **silently truncated** → set num_ctx; ≥64k for agents. :cloud models leave the machine.

LLAMA.CPP · VLLM · SGLANG

```
llama serve -hf bartowski/Meta-Llama-3.1-8B-Instruct-GGUF:Q4_K_M -c 32768 -np 4 -nvl all --alias llama3.1-8b --api-key "$KEY"
# -c shared by -np slots · --jinja default on
# --spec-type ngram-simple | draft-eagle3
```

```
vllm serve meta-llama/llama-3.1-8B-Instruct \
--max-model-len 32768 --tensor-parallel-size 2 \
--enable-auto-tool-choice --tool-call-parser P
sglang serve --model-path <hf-model> --port 30000
```

Bench: vllm bench serve · GuideLLM · sglang.bench_serving

ENDPOINTS & PORTS

Engine	OpenAI base	Anthropic
Ollama	:11434/v1	:11434
llama.cpp	:8080/v1	:8080
vLLM · NIM	:8000/v1	vLLM :8000
SGLang	:30000/v1	:30000
LM Studio	:1234/v1	:1234
Jan	:1337/v1	—
DMR	:12434/engines/v1	:12434
LocalAI · MLX	:8080/v1	LocalAI
LiteLLM	:4000/v1	:4000

Anthropic shape = POST /v1/messages. Health: /health (llama.cpp, vLLM), /v1/health/ready (NIM).

LITELLM GATEWAY

```
model_list:
- model_name: docs-chat
  litellm_params:
    model: ollama_chat/llama3.1:8b
    api_base: http://ollama:11434
    - model_name: claude-sonnet
      litellm_params:
        model: anthropic/claude-sonnet-5
      api_key: os.environ/ANTHROPIC_API_KEY
router_settings:
  fallbacks: [{ "docs-chat": ["claude-sonnet"] }]
general_settings:
  master_key: os.environ/LITELLM_MASTER_KEY
  database_url: os.environ/DATABASE_URL
```

Virtual keys: POST /key/generate · max_budget, budget_duration, rpm_limit (need DB).

GITHUB COPILOT BYOK

- VS Code: **Chat: Manage Language Models** → Add Models
- Ollama: official **Ollama extension** (built-in provider deprecated); VS Code ≥1.127
- Others: **Custom Endpoint** → chatLanguageModels.json, "vendor": "customendpoint", apiType chat-completions | responses | messages, "apiKey": "\${input:key}"
- Policy: *Bring Your Own Language Model Key in VS Code*
- BYOK = chat, agents, utility tasks; **completions, embeddings, semantic search stay on GitHub**

```
export COPILOT_PROVIDER_BASE_URL=http://localhost:11434
export COPILOT_MODEL=qwen3.6 #
TYPE: openai|azure|anthropic
export COPILOT_OFFLINE=true #
tools + streaming, ≥128k ctx
```

CLAUDE CODE

```
ollama launch claude #
or manually:
export ANTHROPIC_BASE_URL=http://localhost:11434
export ANTHROPIC_AUTH_TOKEN=ollama
# required, ignored
ANTHROPIC_DEFAULT_{OPUS,SONNET,HAIKU}.MODEL=my-model
CLAUDE_CODE_ATTRIBUTION_HEADER=0
# keep prefix cache
```

- Supported routes: CLAUDE_CODE_USE_BEDROCK · _VERTEX · _FOUNDRY · gateway (ANTHROPIC_BASE_URL, ANTHROPIC_AUTH_TOKEN / ANTHROPIC_API_KEY, apiKeyHelper, ANTHROPIC_CUSTOM_HEADERS)
- Anthropic **doesn't support routing Claude Code to non-Claude models** through any gateway
- ≥64k context · strong tool calling · vLLM: --served-model-name without "p"

OPEN CODING ASSISTANTS

```
# Continue ~/.continue/config.yaml
models:
- {name: chat, provider: ollama, model: qwen3.6, roles: [chat, edit, apply]}
- {name: fim, provider: ollama, model: qwen2.5-coder:1.5b, roles: [autocomplete]}

# Aider
export OLLAMA_API_BASE=http://127.0.0.1:11434
aider --model ollama_chat/qwen3.6

# OpenCode opencode.json → provider.ollama
"npm": "@ai-sdk/openai-compatible",
"options": {"baseURL": "http://localhost:11434/v1"}
```

Cline: Ollama / LM Studio provider + **Use Compact Prompt**. Only Continue does local **autocomplete**.

OPTIMISATION & OPS

TTFT = t1 - t0 TPOT = (tn - t1)/(n_out - 1)
goodput = requests meeting TTFT & TPOT SLOs / s
E[tokens/step] = (1 - α^(y+1)) / (1 - α)

- Prefill = compute-bound → caching; decode = bandwidth-bound → batching, quant, spec. decoding, GQA
- Stable prompt prefix first → prefix / radix cache hits
- Fit on GPU before adding slots; -c is shared
- Compose: deploy.resources.reservations.devices; cache volumes; long start_period
- Metrics: vllm:time_to_first_token_seconds, vllm:inter_token_latency_seconds, vllm:kv_cache_usage_perc; llama.cpp --metrics
- Cost: C_local = hw/months + P-h·c_kWh + ops vs V_in-p_in + V_out-p_out, capped by goodput

OPEN-WEIGHT LICENCES (AS OF 2026-09-28; "N/V" = NOT VERIFIABLE ON THE VENDOR PAGE — CHECK THE MODEL CARD)

Family	Sizes	Arch	Licence	Family	Sizes	Arch	Licence
Qwen3.6	27B · 35B-A3B	dense · MoE	Apache-2.0	gpt-oss	20b (3.6B act.) · 120b (5.1B act.)	MoE, MXFP4	Apache-2.0
Qwen3.8	27B · 2.4T-A95B	dense · MoE	27B Apache-2.0 · 2.4T custom	Llama 4	Scout 17B-16E · Maverick 17B-128E	MoE	Llama 4 Community License
Gemma 4	E2B · E4B · 12B · 31B · 26B-A4B	dense · MoE	Apache-2.0	Mistral Small 4 · Ministral 3	119B (6.5B act.) · 14B	MoE · dense	Apache-2.0
DeepSeek V4 · V3.2	Flash · Pro	MoE	MIT	Phi-4 · Devstral 2	Phi-4 family · 24B / 123B	dense	n/v

Permissive (Apache-2.0 / MIT) → ship it · community licences → read the terms · gated = accept on the Hub + HF_TOKEN. Quants inherit the base model's licence.